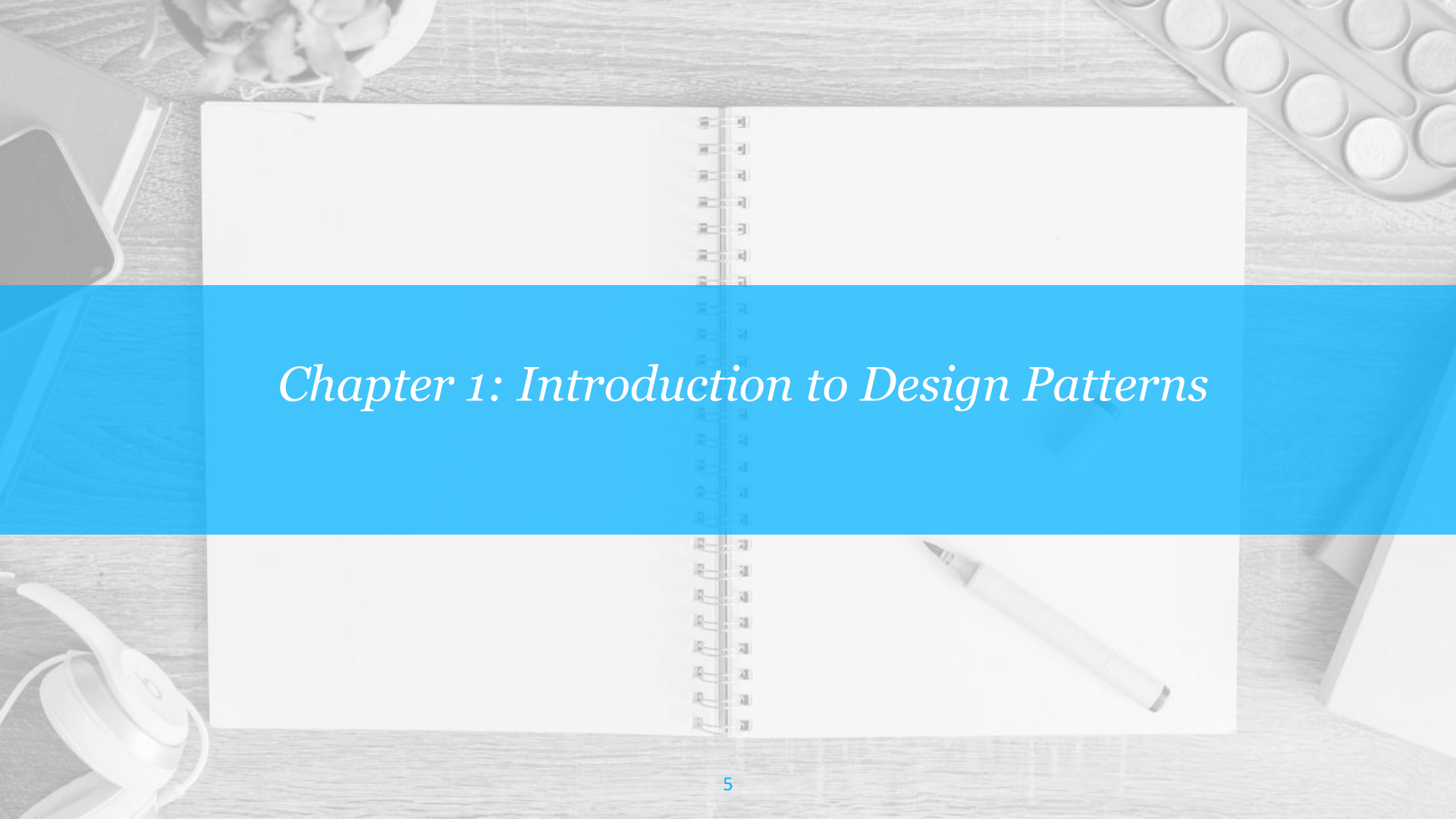


A top-down view of a wooden desk. In the center is a spiral-bound notebook with two blank white pages. A silver pen lies on the bottom right page. In the top right corner, there is a paint palette with several white paint wells. In the top left corner, a small bowl contains some fruit. A white object, possibly a stapler or a small fan, is in the bottom left corner. A blue horizontal band is overlaid across the middle of the notebook.

Chapter 1: Introduction to Design Patterns

1. Lecture Objectives

- By the end of this lecture, students will be able to:
 - Clearly define what a **software design pattern** is
 - Explain **why design patterns exist**
 - Distinguish between **design patterns, algorithms, and frameworks**
 - Understand the **origin and philosophy** of patterns
 - Identify **design problems** that patterns aim to solve
 - Recognize **benefits and dangers** of using patterns

2. Why Do We Need Design Patterns?

2.1 The Reality of Software Development

- Ask students:

→ “Why do large software systems become difficult to maintain over time?”

→ Common answers:

- Code becomes complex
- Changes break existing features
- Adding new features takes too long
- New developers struggle to understand the code

→ Explain:

- These problems do not come from syntax or programming languages — they come from poor design decisions.

2. Why Do We Need Design Patterns?

2.2 Typical Design Problems (Without Patterns)

→ Example Problems

- **Tight Coupling**
 - Classes depend directly on concrete implementations
 - Change in one class → cascade of changes
- **Low Reusability**
 - Code copied and pasted
 - Logic scattered across the system
- **Rigidity**
 - Small changes require major modifications
- **Fragility**
 - Fixing one bug introduces new bugs
- **God Object**
 - One class knows and does everything

2. Why Do We Need Design Patterns?

Simple Code Illustration (Conceptual)

OrderProcessor

└── PaymentProcessor

└── InvoiceGenerator

└── EmailSender

└── DatabaseManager

Explain:

- One class controls everything
- Violates Single Responsibility Principle
- Very hard to test or extend

3. What Is a Software Design Pattern?

3.1 Formal Definition

- A software design pattern is a reusable solution to a commonly occurring problem in software design within a given context.
- Break it down:
 - **Reusable** → Can be applied multiple times
 - **Common problem** → Appears in many systems
 - **Design level** → Not code syntax
 - **Context-dependent** → Not a universal solution